

Regulation Radar for Pet Toys

Time box: 60–90 minutes — please don't spend more

Tools: Any AI coding assistant (Cursor, Claude Code (recommended), Replit, etc.) and any language or stack you like

Deliver: A repo (or zip) we can run, a short README, and a 3–5 minute screen recording walking us through it

The Scenario

We sell pet toys. Different products are subject to different regulations — material safety limits, choking-hazard standards, chemical disclosure rules, labeling requirements — and those rules *change over time*. Today the team tracks this in a spreadsheet that nobody trusts and nobody updates.

We want a small dashboard that (a) shows which regulations currently apply to our pet toys (use BarkBox as an example), sourced from a real link, and (b) is watched by one or more **agents** that periodically re-check the source, detect when something has changed, and update the guidance shown on the dashboard.

This is intentionally too big to finish in the time box, and that's the point. We're evaluating how you **scope**, how you **design the agent loop**, and how you make sensible product calls under time pressure while “vibe coding” with an AI assistant. A small, honest, working slice beats a sprawling unfinished one.

Please respect the time box. Stop at ~90 minutes even if it's not “done.” We'd much rather see one regulation flowing all the way through than a half-built everything. Note in your README what you'd do next.

What to Build

A minimal working slice with three parts. Get something end-to-end running before polishing any one piece.

1. Dashboard — full-stack

A simple UI that lists tracked regulations for pet toys. Each row should show at least: the regulation name, what it applies to, a one-line plain-English guidance summary, a link to the source, a “last checked” timestamp, and a status (e.g. *current* / *changed* / *needs review*). Pull the initial data from the source link rather than hardcoding all of it.

2. The Agent — AI design

Design and wire up an agent that, on demand (a “Run check now” button is fine — no real cron needed), re-fetches the source, compares it to what's stored, decides whether anything materially changed, and writes updated guidance plus a changed / needs-review flag back to the dashboard. Stub the model call if you run low on time, but the **orchestration** — fetch → diff → decide → update — should be real and runnable.

3. Product judgment

Make and state your scope cuts in the README. When the agent thinks something changed, what does the user see and what can they do about it? How do you avoid silently overwriting human-reviewed guidance? You won't build all of this — write down what you'd build and why.

Suggested Source Links

Pick one. Don't get stuck on parsing — a rough extract is fine.

- A page from the **eCFR** (e.g. CPSC regulations under Title 16) — clean structure and an API.
- A **California Prop 65** chemical list page.
- An **ASTM / consumer-product safety** standards summary page.

If a source is hard to fetch or render, note that in your README and mock the fetched payload. We care about the loop, not the scraper.

What to Submit

- **The code** — a public repo link or a zip. Include exact run instructions (a single command if you can).
- **A short README** — how to run it, your scope cuts and assumptions, how the agent decides “something changed,” and what you’d build with more time.
- **A 3–5 minute screen recording** (Loom or similar) — demo the dashboard, run a check, and talk through one design decision you’re proud of and one shortcut you took.

How We’ll Evaluate

Area	What good looks like
Agent design 35%	A clear, real fetch → diff → decide → update loop. Sensible “did this change materially?” logic. Thoughtful prompt and tool boundaries. Guards against bad or empty fetches.
Full-stack execution 30%	It actually runs from your instructions. Reasonable data model, source ingestion, and a readable UI. Clean enough that a teammate could extend it.
Product judgment 20%	Smart scope cuts, clearly written down. Good handling of the human-in-the-loop “don’t overwrite reviewed guidance” question.
Working with AI 15%	Effective, legible use of the coding assistant — visible in the code and the recording. Knows when to trust it and when to correct it.

Stretch Ideas (only if you have time to spare — most won’t)

- A second agent that *classifies* which products a changed rule affects.
- A diff view showing old vs. new guidance.
- An approval step so a human confirms agent-proposed changes before they go live.
- Notifications (“3 regulations changed since you last looked”).

Use your AI assistant the way you actually would on the job — we encourage it. We’re evaluating you *with* the tool, not testing whether you can work without it. Questions about scope? Make a reasonable assumption, write it down, and keep moving.